

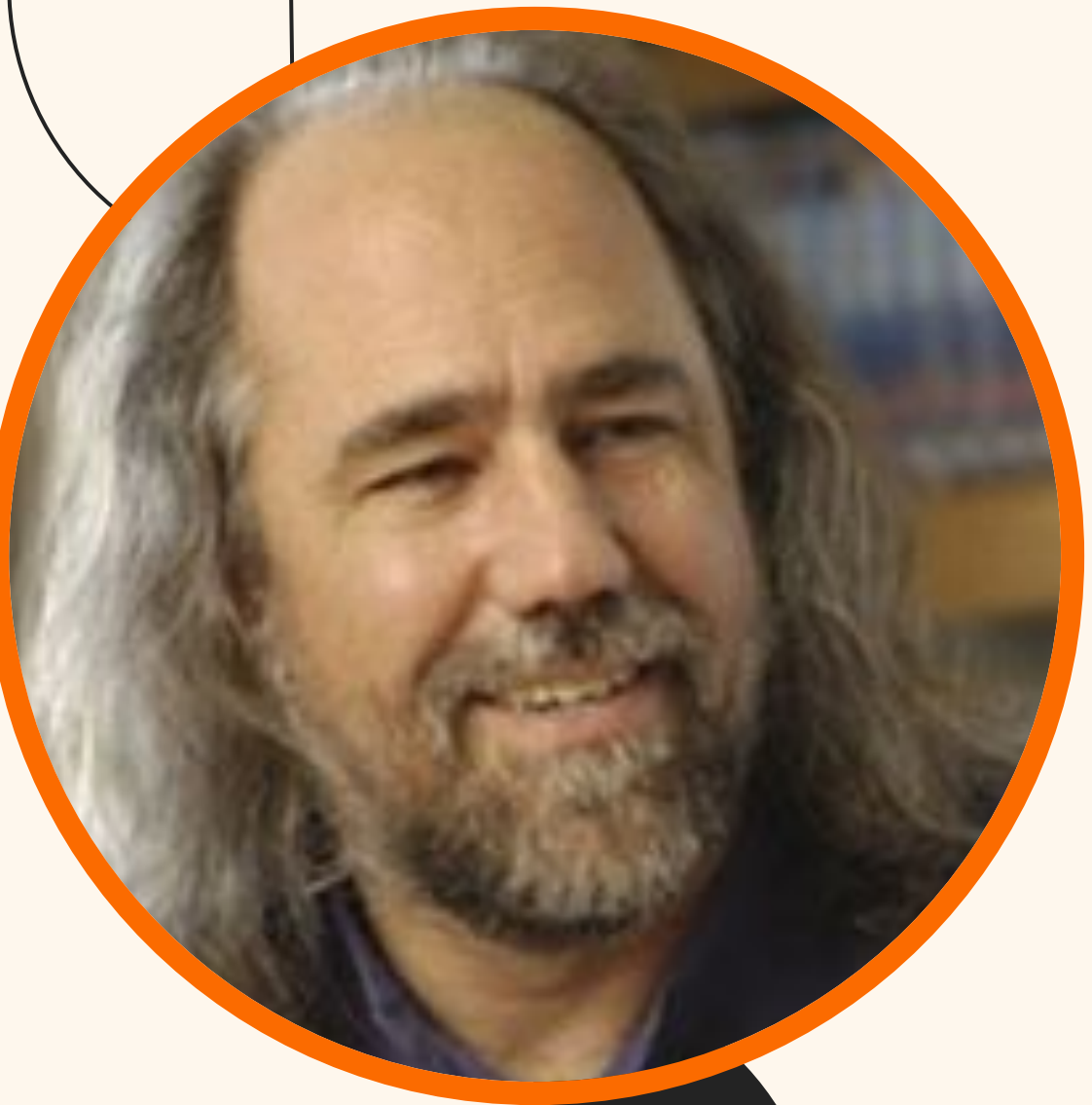


Programação Orientada a Objetos

Abstração / Encapsulamento

Maurício da Motta Braga
mmb2@cesar.school





"The entire history of software engineering is that of the rise in levels of abstraction."

Grady Booch.

P OO – quatro pilares

Abstração

Encapsulamento

Herança

Polimorfismo



POO – quatro pilares

Abstração



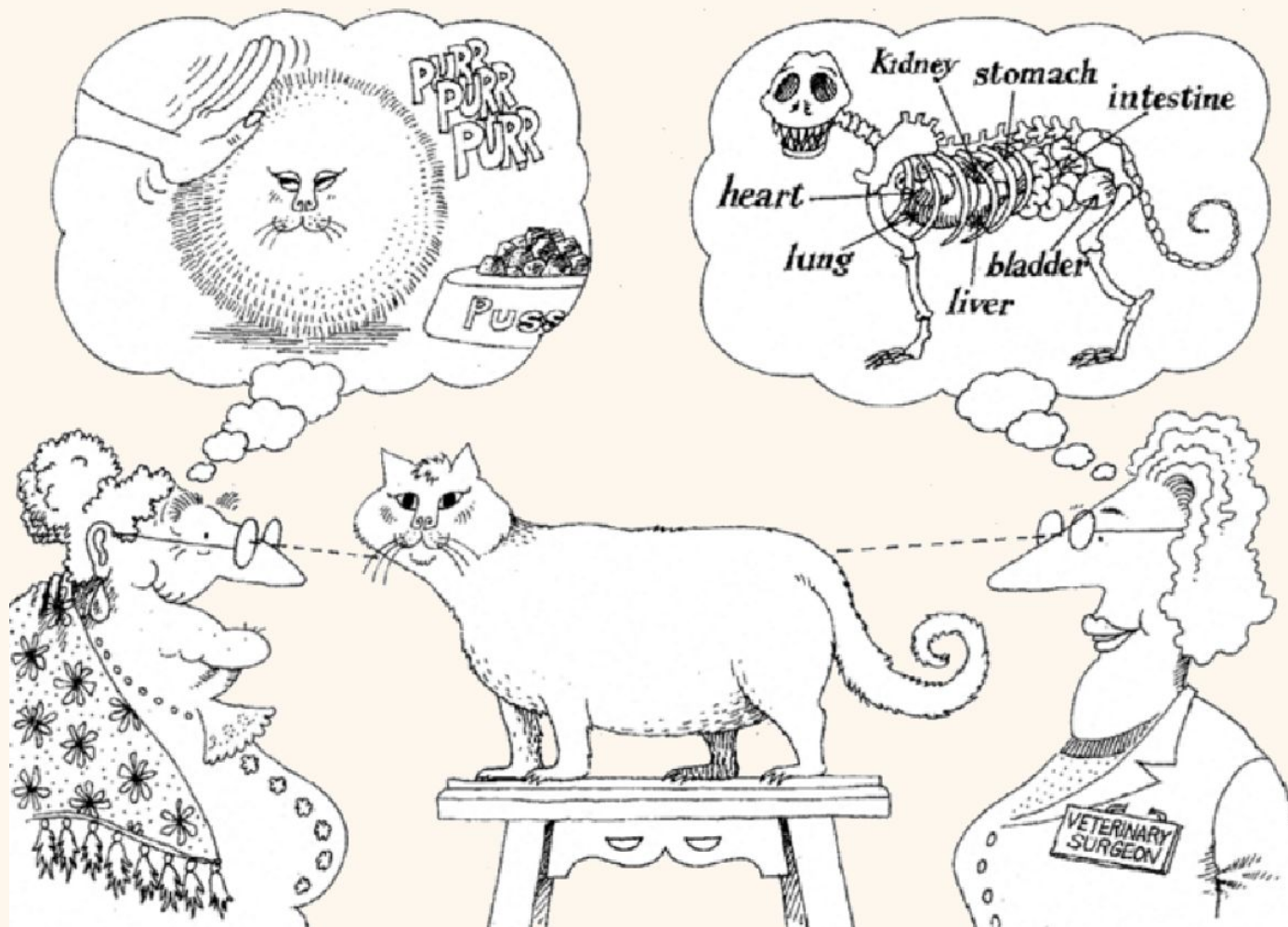
```
graph TD; A[Abstração] --> B[Encapsulamento]; B --> C[Herança]; C --> D[Polimorfismo];
```

Encapsulamento

Herança

Polimorfismo

Abstração



Foco nas características essenciais de um objeto, do ponto de vista do usuário.

Abstração

Uma das formas pelas quais podemos lidar com a complexidade.

Surge do reconhecimento das semelhanças entre objetos e da decisão de focar nessas semelhanças e ignorar por enquanto as diferenças entre eles, focando na versão “generalizada” do objeto.

Um conceito se qualifica como uma abstração somente se puder ser descrito, compreendido e analisado independentemente do mecanismo que eventualmente será usado para realizá-lo.

Abstração

Permite que, ao modelar um objeto, se concentre no que um objeto faz, em vez de como ele faz.

Busca caracterizar um objeto considerando as responsabilidades desse objeto (a partir da perspectiva dos serviços que ele presta a outros objetos).

É o ato de representar entidades do mundo real de forma simplificada, focando apenas no que é relevante para o contexto em questão.

Exemplo

Abstração: MediaPlayer
Características: Nome da música, formato, localização...
Responsabilidades: Reproduzir música, interromper execução, avançar, retroceder...

Construir um objeto para tocar música pode ser complicado (vários formatos de música (MP3, Flac, Midi, Wave etc), complexidade dos formatos etc).

|

Podemos ignorar por enquanto esses detalhes e focar nas características essenciais do objeto ao modelá-lo. Essas características essenciais são ter métodos para reproduzir música (PLAY), interromper a execução da música (STOP) etc, sem se preocupar em como esses métodos serão implementados.

Exemplo

Abstração: MediaPlayer
Características: Nome da música, formato, localização...
Responsabilidades: Reproduzir música, interromper execução, avançar, retroceder...

Dessa forma, utilizar o objeto mediaPlayer (chamando seus métodos) pode ser bem simples, bastando chamar play, stop, pause etc sem o usuário precisar saber os detalhes de como esses métodos funcionam.

Alguns benefícios do uso da abstração

Facilita a compreensão do código, pois permite que os desenvolvedores vejam o problema de forma mais simples e clara.

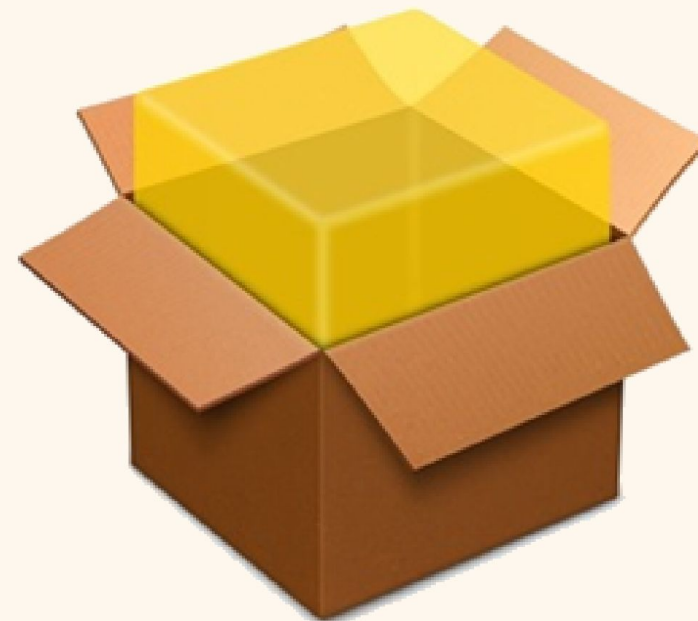
Melhora o reaproveitamento do código, pois permite que os desenvolvedores criem código modular e reutilizável.

Redução do número de erros.



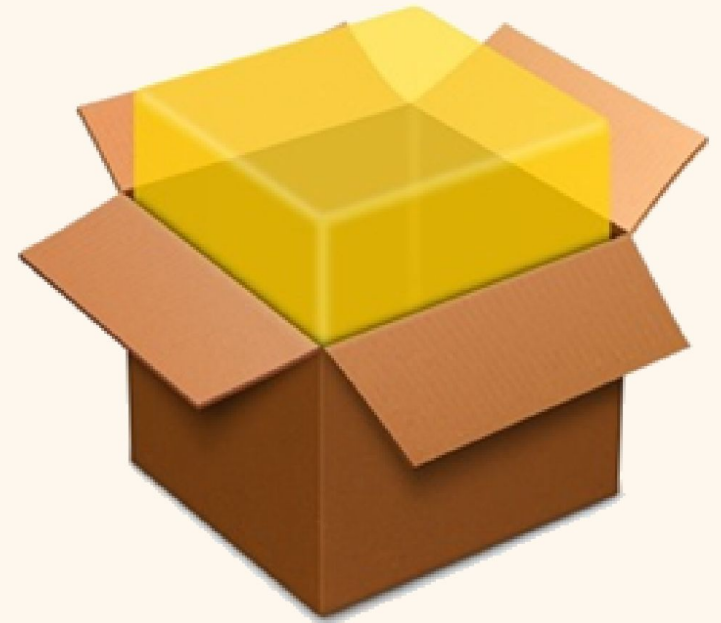
Pacotes

- ▶ Representam um agrupamento de classes correlatas.
- ▶ Pastas e subpastas são criadas quando o nome do pacote é formado por palavras separadas por ponto.



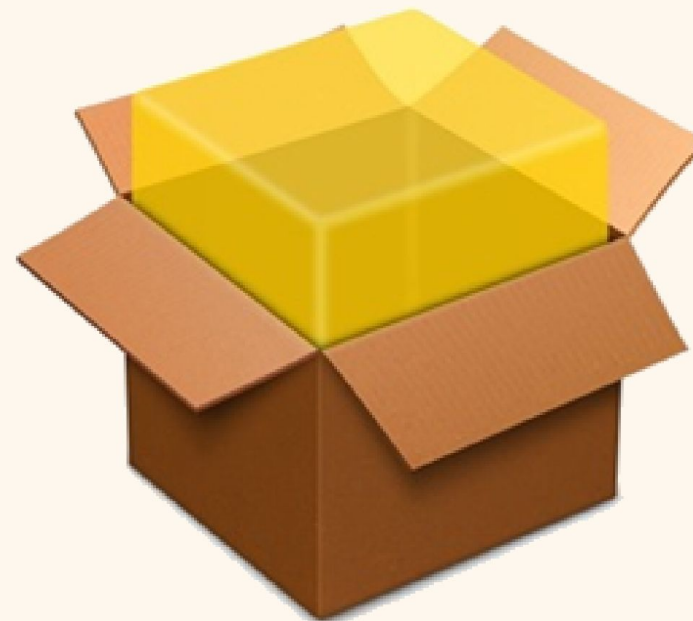
Pacotes

- ▶ Critérios para se definir pacotes:
 - ▶ Por funcionalidade
 - ▶ Por estereótipo
 - ▶ Por camada
 - ▶ Por conceito



Pacotes

- ▶ Como definir pacotes em JAVA:
 - ▶ Em uma classe (.java), declarar o pacote em que a classe se encontra, usando a palavra reservada **package**.
 - ▶ Nas IDEs de desenvolvimento, criam-se as estruturas de pacote pré-definidas, e depois criam-se as classes já localizadas nos seus respectivos pacotes.



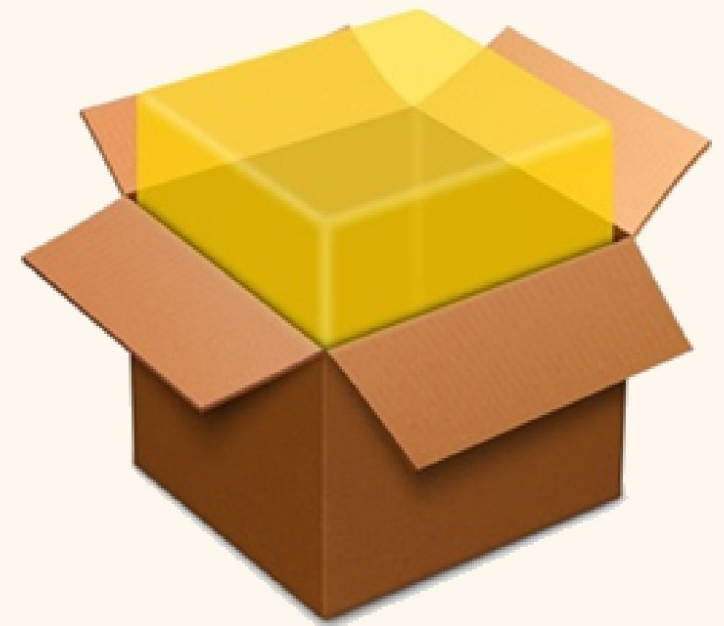
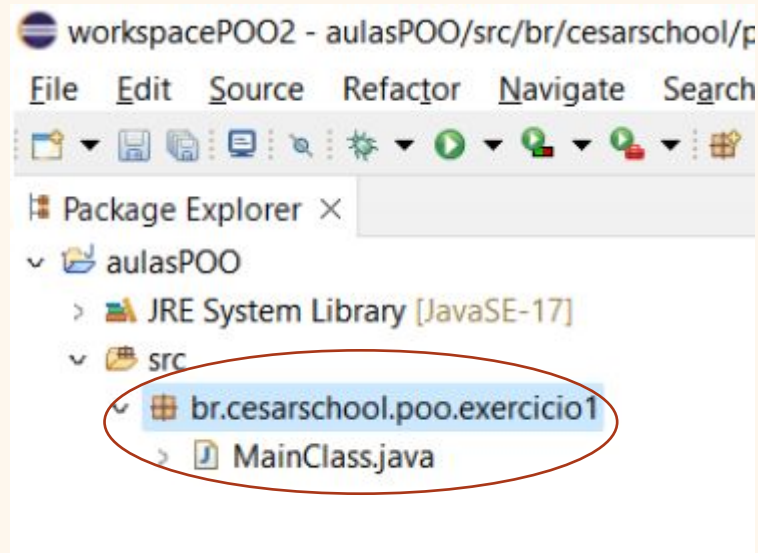
Pacotes

```
package br.cesarschool.poo.exercicio1;
```

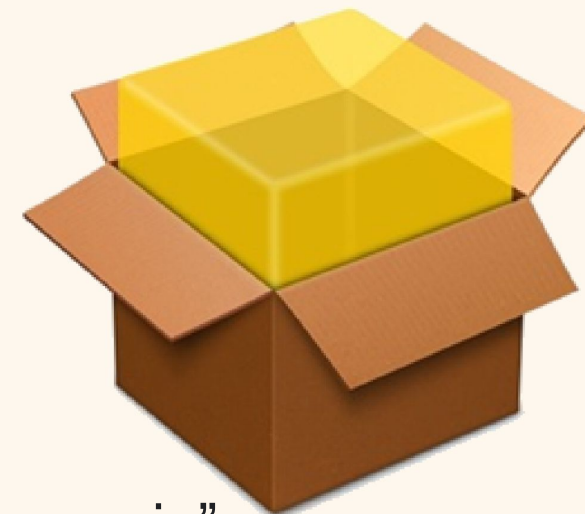
```
public class MainClass {
```

```
    public static void main(String[] args) {  
        System.out.println("ALO MUNDO!!");  
    }
```

```
}
```



Pacotes



- ▶ Padrão de nomenclatura:

- ▶ Tudo escrito em letras minúsculas
- ▶ Sem espaços, hifens ou underscores
- ▶ Pontos entre palavras normalmente representam uma “hierarquia”

- ▶ Em muitos projetos corporativos se utiliza a seguinte convenção:

`país.tipo de
organização.organização.departamento.sistema."caminho interno"`

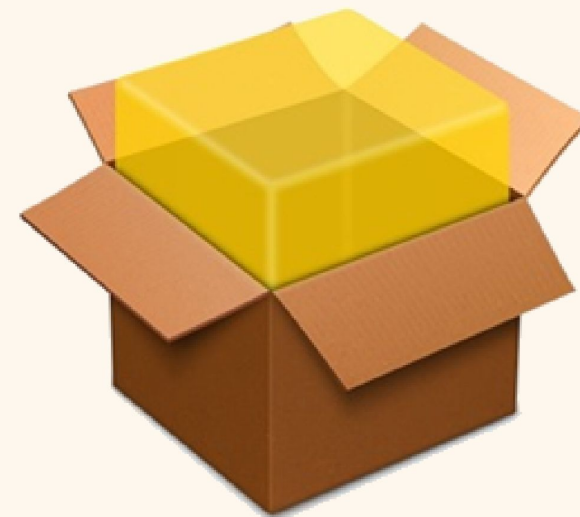
Exemplos:

`br.gov.sefazpe.dti.spo.cadastro.produto`

`us.edu.mit.sdd.rds.mediator`

Pacotes

- ▶ Quando uma classe usa classes de pacotes diferentes da dela, é necessário declarar as classes usadas através da diretiva **import**.
- ▶ A seção de imports fica entre a declaração **package** e a declaração da primeira classe do arquivo.
- ▶ Não é necessário “importar” as classes do “core” do JAVA, elas já são importadas automaticamente.



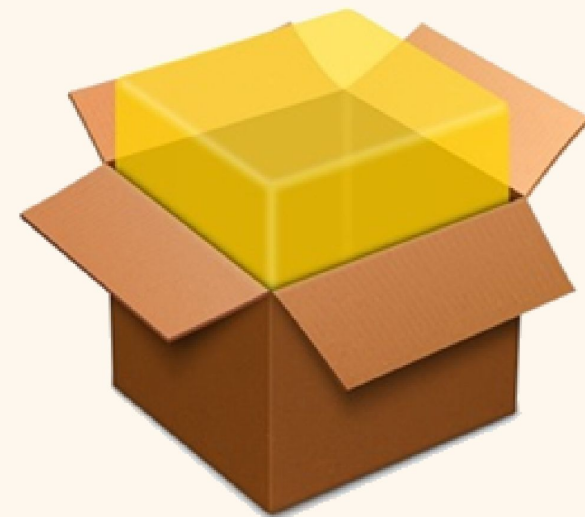
Pacotes

- ▶ As IDE's já colocam os imports automaticamente, à medida que forem feitas referências a classes na implementação.
- ▶ Pode se importar apenas uma classe (1ª forma) ou todas as classes de um pacote (2ª forma).
- ▶ As boas práticas de programação recomendam importar classe a classe.

```
import <nome do pacote>.<nome da classe>
```

OU

```
import <nome do pacote>.*
```



P OO – quatro pilares

Abstração



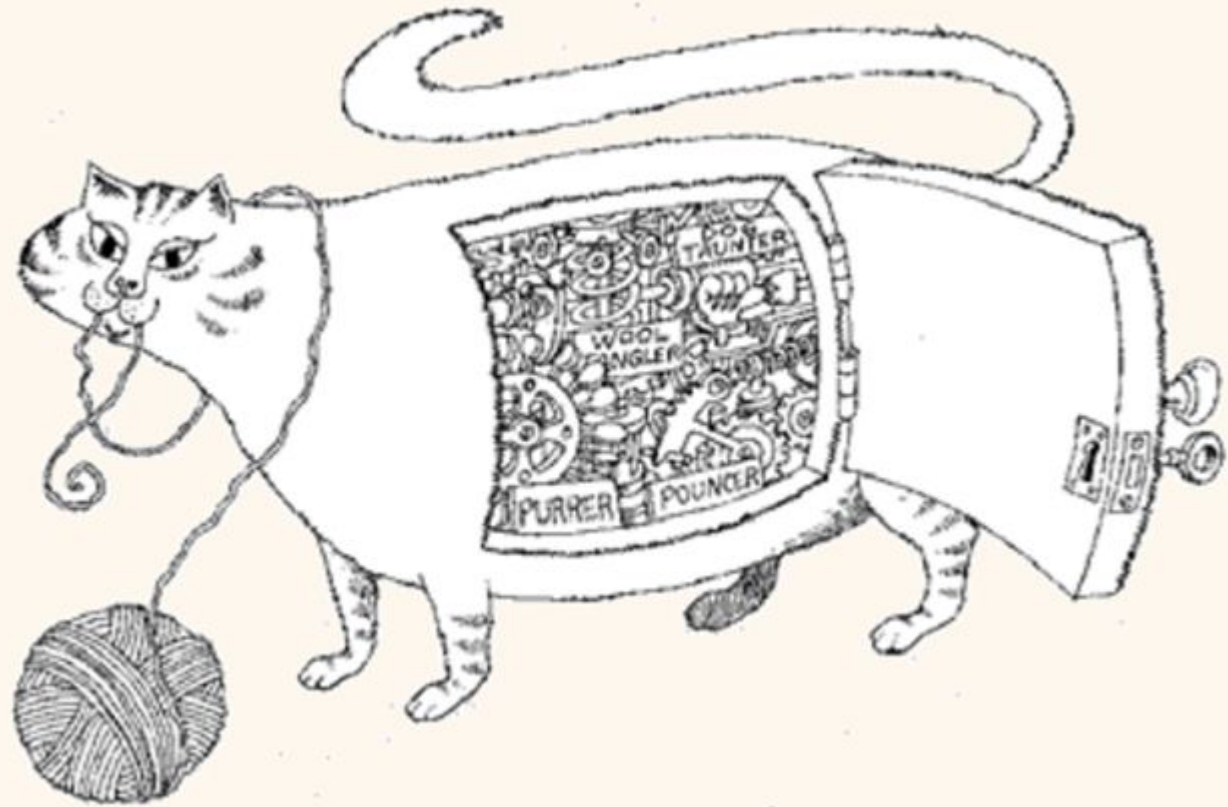
```
graph TD; A[Abstração] --> B[Encapsulamento]; B --> C[Herança]; C --> D[Polimorfismo];
```

Encapsulamento

Herança

Polimorfismo

Encapsulamento



Separa a interface de um objeto dos detalhes do seu funcionamento interno.

Encapsulamento

- ▶ Técnica utilizada para não expor detalhes internos de uma classe para os seus usuários, restringindo o acesso a eles de forma a garantir que sejam protegidos e usados de forma correta.
 - ▶ O acesso aos atributos deve ser controlado através de métodos de acesso.
 - ▶ Os métodos disponíveis para acesso fora de uma classe devem ser apenas os que definem a “interface funcional” necessária desta classe para outras.
-

Interface

- ▶ Lista de serviços oferecidos por um componente
 - ▶ Funciona como um “contrato” que o componente estabelece com o mundo exterior que define exatamente como este pode fazer uso do componente.
 - ▶ Funciona como um “painel de controle” do componente.
-

Interface

- ▶ Formada por métodos e atributos públicos.
 - ▶ Ausência de informações sobre como o trabalho é realizado.
 - ▶ Liberdade para modificar o funcionamento do componente sem “quebra de contrato” com o mundo exterior.
-

Exemplo Encapsulamento

```
class Cliente {  
    private double credito;  
    private String nome;  
    public String getNome() {  
        return nome;  
    }  
    public void imprimaDadosCliente(Cliente c) {  
        imprimir(c.nome, c.credito);  
    }  
    private void imprimir(String nome, double credito) {  
        System.out.println("Cliente: " + nome + " Crédito: R$ " + credito);  
    }  
}
```

Interface

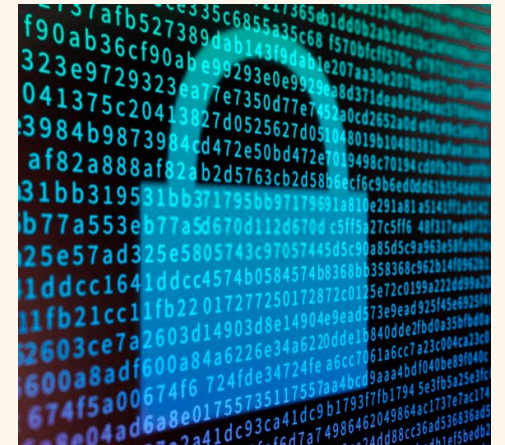
```
public String getNome()
```

```
public void imprimaDadosCliente(Cliente c)
```

**Note que o método privado
imprimir não faz parte da
interface!**

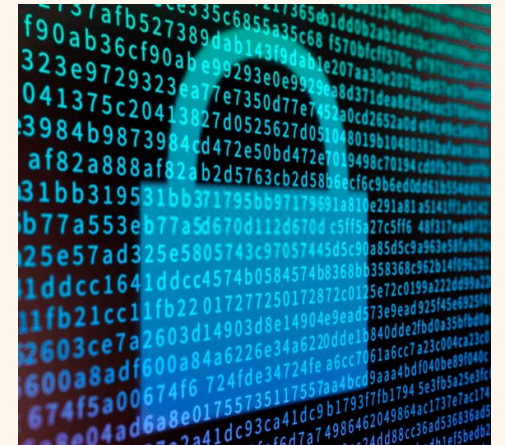
Modificadores de acesso

- ▶ Definem o nível de acesso a classes, métodos e atributos por outras classes que desejam utilizá-los.
- ▶ Como as classes em JAVA são organizadas em pacotes e hierarquias, os níveis de acesso são determinados em função destes dois elementos, além de, obviamente, a própria classe.
- ▶ A tentativa de acesso a um elemento “proibido” pelos modificadores de acesso provoca erro de compilação



Modificadores de acesso

- ▶ Níveis de visibilidade:
 - ▶ **Público**: qualquer classe pode acessar um elemento público. Aplicável a classes, métodos e atributos. Palavra reservada: **public**.
 - ▶ **Protegido**: qualquer classe de um mesmo pacote ou de mesma sub hierarquia pode acessar um elemento protegido. Aplicável a métodos e atributos. Palavra reservada: **protected**
 - ▶ **Default**: qualquer classe de um mesmo pacote pode acessar um elemento default. Aplicável a classes, métodos e atributos. Palavra reservada: não se coloca nada
 - ▶ **Privado**: só a classe onde está definido um elemento privado pode acessá-lo. Aplicável a métodos e atributos. Palavra reservada: **private**



Modificadores de acesso

Acesso **sem**
restrições

public

Restringe o
acesso à **classe**,
subclasse e
pacote

protected

Restringe acesso
ao **mesmo**
pacote

default

Restringe o
acesso **apenas** à
classe

private

	classe	pacote	subclasses	universo
private	X			
default	X	X		
protected	X	X	X	
public	X	X	X	X

Modificadores de acesso

- ▶ Boas práticas de uso dos modificadores:
 - ▶ Atributos via de regra são definidos como privados.
 - ▶ Se precisarem ser expostos, usam-se métodos “acessores”.
 - ▶ Torne privado tudo o que for possível.
 - ▶ Exponha o mínimo necessário.



Métodos get e set

- ▶ Definem-se os métodos “acessores” (getters e setters) como public, protected ou default, com o atributo que eles dão acesso como private.
- ▶ Para atributos do tipo boolean, é comum se usar isNomeDoAtributo em vez de getNomeDoAtributo
- ▶ Quando um atributo não pode ou não deve ser modificado por outras classes, mas pode ser lido, define-se apenas um método “acessor” (o get).
- ▶ Sintaxe:

```
<visibilidade> <tipo do atributo> getNomeDoAtributo() {  
    return this.nomeDoAtributo;  
}
```

```
<visibilidade> void setNomeDoAtributo(<tipo atributo> nomeDoAtributo) {  
    this.nomeDoAtributo = nomeDoAtributo;  
}
```

Métodos de acesso

COM ENCAPSULAMENTO

SEM ENCAPSULAMENTO

```
package br.cesarschool.poo.exercicio1;
public class ExemploSetGet {
    public int idade;
    public String nome;
    public double renda;
}
```



```
package br.cesarschool.poo.exercicio1;
public class ExemploSetGet2 {
    private int idade;
    private String nome;
    private double renda;
    public int getIdade() {
        return idade;
    }
    public void setIdade(int idade) {
        this.idade = idade;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public double getRenda() {
        return renda;
    }
    public void setRenda(double renda) {
        this.renda = renda;
    }
}
```

Exercício

- Crie uma classe que represente um círculo.
- Esta classe deve utilizar encapsulamento, conter apenas uma variável (**raio**), um método **getArea** para calcular (e retornar) a área e outro método **getComprimento** para calcular e retornar o comprimento.
- Adicione também um método **toString()** que retorna uma string com todas as informações do círculo (valor das variáveis internas, área e comprimento).

Dados:

Comprimento = $2 * \text{Pi} * \text{Raio}$

Área = $\text{Pi} * \text{Raio}^2$

- Dica: Use `Math.PI`.



Solução

```
public class Circulo {  
    private double raio;  
    public Circulo (double raio) {  
        this.raio = raio;  
    }  
    public double getRaio() {  
        return raio;  
    }  
    public void setRaio(double r) {  
        raio = r;  
    }  
}
```

Solução

```
public double getArea() {  
    return Math.PI*raio*raio;  
}  
public double getComprimento() {  
    return 2*Math.PI*raio;  
}  
public String toString() {  
    return "Círculo [raio = "+raio + ", area = " + getArea() + ", comprimento = " +  
    getComprimento();  
}  
}
```

Modificador “static”



A palavra reservada **static**, quando colocada na frente da definição de um atributo ou de um método, significa “de classe”.



Atributos e métodos static pertencem à classe, e não a objetos instanciados a partir desta.



Exemplo

```
public class ExemploStatic {
    private double valor;
    private static String rotulo;

    public double getValor() {
        return this.valor;
    }
    public void setValor(double valor) {
        this.valor = valor;
    }
    public static void setRotulo(String rot) {
        rotulo = rot;
    }
    public static String getRotulo() {
        return rotulo;
    }
}
```

```
public class Exemplo {

    public static void main(String[] args) {
        ExemploStatic ex1 = new ExemploStatic();
        ExemploStatic ex2 = new ExemploStatic();
        ex1.setValor(100.00);
        ex2.setValor(50.00);
        System.out.println("ex1 valor: " + ex1.getValor());
        System.out.println("ex2 valor: " + ex2.getValor());
        ex1.setRotulo("Abacate");
        ex2.setRotulo("Laranja");
        System.out.println("ex1 rótulo: " + ex1.getRotulo());
        System.out.println("ex2 rótulo: " + ex2.getRotulo());
    }
}
```

O modificador "static"

- Ao contrário do exemplo visto, não é recomendável referenciar atributos e métodos estáticos a partir de variáveis que apontam para instâncias de objetos.
- Em vez disso, usa-se, SEMPRE:

<NOME DA CLASSE>.<atributo ou método static>.

- Em uma mesma classe:
 - Métodos static NÃO acessam atributos e métodos não static, se um programador tentar fazer isto, o código não compila.
 - Métodos não static acessam atributos e métodos static e não static.
-

O modificador "static"

- Métodos e atributos static são ditos métodos e atributos de classe.
 - Métodos e atributos não static são ditos métodos e atributos de instância, ou de objeto.
 - O modificador static não invalida o efeito dos outros modificadores já estudados (public, protected, default e private).
-

0 modificador de invariância “final”



A palavra reservada **final** significa invariância, imutabilidade



Quando aplicado a um atributo, não permite que ele seja alterado



Quando aplicado a uma classe, não permite que ela seja herdada, ou seja, que ela tenha sub hierarquia (vamos ver isso mais na frente)



Quando aplicado a um método, não permite que ele seja sobrescrito (vamos ver isso mais na frente)

Em todos os casos acima, “não permitir”
significa que, se o programador tentar o contrário,
o código não compila

O modificador de invariância “final”

- Constantes em JAVA
 - Normalmente, são definidas como “static final” no escopo de atributos de uma classe
 - Muito eventualmente são definidas como “final”, também no escopo de atributos de uma classe
 - Podem ser definidas no escopo de blocos de código, mas não se trata de uma prática muito comum
 - Quando atributos, têm o modificador de visibilidade que for mais conveniente e pertinente
 - Não podem estar, em linhas subsequentes à sua definição e inicialização, no lado esquerdo de uma operação de atribuição, sob pena do código não compilar
-

Algumas vantagens do uso do encapsulamento

Torna o software mais flexível, fácil de modificar e de criar novas implementações.

Facilita a organização do código e aumenta a possibilidade de reuso de componentes.

O uso do encapsulamento ajuda a reduzir o acoplamento entre partes de um sistema.



**Slides adaptados a partir
do material produzido pelo
prof. Eduardo Calábria**

egc2@cesar.school





**MAURÍCIO DA
MOTTA BRAGA**



mmb2@cesar.school

